

JobSheet - Week 6 – Java Stream API

Nama : Raza Mulki Firdaus
NIM : 251524120
Kelas : 1D
Repo GitHub : <https://github.com/zaaraza/Teknik-Pemrograman->

Instruksi Pengerjaan:

1. Kerjakan 3 soal di bawah ini dengan melengkapi setiap kolom jawaban yang disediakan pada jobsheet ini.
2. Jawaban setiap soal mencakup source code, screenshot hasil dari program yang ditampilkan full screen termasuk taskbar (tambahkan beberapa screenshot jika diperlukan), penjelasan permasalahan dan solusi yang dihadapi, nama teman yang membantu memecahkan masalah (opsional).
3. Dikumpulkan pada Assignment Classroom sesuai dengan deadline yang tertera pada assignment tersebut.
4. Format penamaan file jobsheet: W6_P_<Kelas 1X>_<3 Digit_NIM_Terakhir>.docx/pdf. Contoh: W6_P_1B_001.docx/pdf.
5. Submit semua jawaban dalam bentuk file java pada repository GitHub masing-masing.

No. 1 Java Stream Sorting

Soal Praktikum
Instruksi Lengkapilah bagian kosong pada cuplikan kode di bawah ini dengan ketentuan sebagai berikut: 1. Bagian [1]: Panggil metode untuk mengubah ArrayList menjadi Stream. 2. Bagian [2]: Gunakan metode pengurutan (intermediate operation) yang menerima lambda expression. 3. Bagian [3]: Panggil metode getter yang sesuai untuk membandingkan nama antar objek. 4. Bagian [4]: Gunakan operasi terminal untuk merubah aliran data kembali menjadi objek List. 5. Bagian [5]: Gunakan utility class dan metode yang tepat untuk mengumpulkan hasil akhir ke dalam format List.
Source Code

```

package JavaProblems;

import java.util.ArrayList;
import java.util.List;
import java.util.stream.Collectors;

public class ProblemOne {
    public static void main(String[] args) {
        ArrayList<Employee> list = new ArrayList<>();

        list.add(new Employee("John", 50000));
        list.add(new Employee("Rob", 70000));
        list.add(new Employee("Bob", 40000));
        list.add(new Employee("Alice", 10000));

        // Melakukan proses filtering/sorting menggunakan Stream API
        List<Employee> sortedEmp = list.[1]_____ // Ubah list ke stream
            // Mengurutkan employee berdasarkan nama
            .[2]_____((e1, e2) -> e1.[3]_____.compareTo(e2.getName()))
            // Mengumpulkan hasil akhir
            .[4]_____([5]_____.toList());

        for(Employee e : sortedEmp){
            System.out.println(e);
        }
    }
}

// Buatlah class Employee disertai dengan getter getName() dan getSalary(),
// termasuk attribute dan method lainnya jika diperlukan!

```

Output yang Diharapkan

```

Employee{name='Alice', salary=50000}
Employee{name='Bob', salary=70000}
Employee{name='John', salary=10000}
Employee{name='Rob', salary=40000}

```

Source Code

```

1 package JavaProblems;
2
3 import java.util.ArrayList;
4 import java.util.List;
5 import java.util.stream.Collectors;
6
7 class Employee {
8     private String name;
9     private int salary;
10
11     public Employee(String name, int salary) {
12         this.name = name;
13         this.salary = salary;
14     }
15
16     public String getName() {
17         return name; }
18     public int getSalary() {
19         return salary; }
20
21     @Override
22     public String toString() {
23         return "Employee{Name='" + name + "', Salary=" + salary + "}";
24     }
25 }
26
27 public class ProblemOne {
28     public static void main(String[] args) {
29         ArrayList<Employee> list = new ArrayList<>();
30         list.add(new Employee("John", 50000));
31         list.add(new Employee("Rob", 70000));
32         list.add(new Employee("Bob", 40000));
33         list.add(new Employee("Alice", 10000));
34
35         List<Employee> sortedEmp = list.stream() // [1]
36             .sorted((e1, e2) -> e1.getName().compareTo(e2.getName())) // [2] & [3]
37             .collect(Collectors.toList()); // [4] & [5]
38
39         for(Employee e : sortedEmp){
40             System.out.println(e);
41         }
42     }
43 }

```

Screenshot Hasil

```

Employee{Name='Alice', Salary=10000}
Employee{Name='Bob', Salary=40000}
Employee{Name='John', Salary=50000}
Employee{Name='Rob', Salary=70000}

```

Penjelasan

Soal ini bertujuan untuk mengurutkan daftar Employee berdasarkan atribut Nama dengan menggunakan Java Stream API.

- Konversi ke Stream, `ArrayList<Employee>` diubah menjadi aliran data menggunakan metode `.stream()` agar bisa diproses secara deklaratif.
- Intermediate Operation (Sorting), digunakannya metode `.sorted()` yang menerima lambda expression `(e1, e2) -> e1.getName().compareTo(e2.getName())`. Disini `e1` dan `e2` dibandingkan menggunakan metode `getName()` untuk mengambil string nama. Setelah itu digunakan metode `compareTo()` untuk sorting dari A ke Z.

- Terminal Operation: Setelah data terurut di dalam stream, digunakan metode `.collect(Collectors.toList())` untuk mengubah kembali aliran data tersebut menjadi objek `List<Employee>` yang baru.
- Output Adalah daftar karyawan beserta gaji yang sudah tersusun rapi berdasarkan nama mereka.

Nama Teman Hal yang Dibantu (Opsional)

-

No. 2 Multi-Level Sorting dengan Stream

Soal Praktikum

Skenario

Anda diminta untuk mengurutkan daftar objek Student. Aturan pengurutannya adalah:

1. Urutkan berdasarkan Nama secara alfabetis (tanpa mempedulikan huruf besar/kecil).
2. Jika ada nama yang sama, maka urutkan berdasarkan ID dari yang terkecil ke terbesar.

Instruksi Pengerjaan

Lengkapilah bagian kosong pada kode di bawah ini dengan ketentuan:

1. Bagian [1]: Panggil fungsi untuk memulai operasi Stream pada koleksi.
2. Bagian [2]: Gunakan metode Stream untuk melakukan pengurutan elemen.
3. Bagian [3]: Gunakan metode dari kelas wrapper Integer untuk membandingkan dua nilai ID secara aman.
4. Bagian [4]: Gunakan metode String yang tepat untuk membandingkan dua string tanpa memperhatikan case (huruf besar/kecil) untuk menghasilkan nilai integer perbandingan.
5. Bagian [5]: Panggil operasi terminal untuk mengumpulkan hasil akhir ke dalam sebuah List.

Instruksi

```
package JavaProblems;
```

```
import java.util.Arrays;
```

```
import java.util.List;
```

```
import java.util.stream.Collectors;
```

```
public class ProblemTwo {
```

```
    public static void main(String[] args) {
```

```
        List<Student> students = Arrays.asList(
            new Student(101, "Joe"),
            new Student(103, "Zulkifli"),
            new Student(102, "Riza"),
            new Student(104, "Alice"),
            new Student(105, "Joshua")
        );
```

```
        List<Student> ans = students.[1]_____ // Membuka stream
            .[2]_____((s1, s2) -> {    // Melakukan pengurutan
```

```
            // Logika: Jika nama sama, bandingkan ID
```

```
            if (s1.getName().equalsIgnoreCase(s2.getName())) {
```

```
                return [3]_____.compare(s1.getId(), s2.getId());
```

```

    }
    // Jika nama berbeda, bandingkan Nama
    else {
        return s1.getName().[4]_____ (s2.getName());
    }
})
.[5]_____ (Collectors.toList()); // Mengumpulkan hasil

for (Student student : ans) {
    System.out.println(student);
}
}
}

// Buatlah class Student disertai dengan attribute dan method lainnya yang
diperlukan!

```

Output yang Diharapkan

```

Student{id=104, name='Alice'}
Student{id=101, name='Joe'}
Student{id=105, name='Joshua'}
Student{id=102, name='Riza'}
Student{id=103, name='Zulkifli'}

```

Source Code

```

1 package JavaProblems;
2
3 import java.util.Arrays;
4 import java.util.List;
5 import java.util.stream.Collectors;
6
7 class Student {
8     private int id;
9     private String name;
10
11     public Student(int id, String name) {
12         this.id = id;
13         this.name = name;
14     }
15
16     public int getId() { return id; }
17     public String getName() { return name; }
18
19     @Override
20     public String toString() {
21         return "Student{id=" + id + ", name='" + name + "'}";
22     }
23 }
24
25 public class ProblemTwo {
26     public static void main(String[] args) {
27         List<Student> students = Arrays.asList(
28             new Student(101, "Joe"),
29             new Student(103, "Zulkifli"),
30             new Student(102, "Riza"),
31             new Student(104, "Alice"),
32             new Student(105, "Joshua")
33         );
34
35         List<Student> ans = students.stream() // [1]
36             .sorted((s1, s2) -> { // [2]
37                 if (s1.getName().equalsIgnoreCase(s2.getName())) {
38                     return Integer.compare(s1.getId(), s2.getId()); // [3]
39                 } else {
40                     return s1.getName().compareToIgnoreCase(s2.getName()); // [4]
41                 }
42             })
43             .collect(Collectors.toList()); // [5]
44
45         for (Student student : ans) {
46             System.out.println(student);
47         }
48     }
49 }

```

Screenshot Hasil

```

Student{id=104, name='Alice'}
Student{id=101, name='Joe'}
Student{id=105, name='Joshua'}
Student{id=102, name='Riza'}
Student{id=103, name='Zulkifli'}

```

Penjelasan

Program ini mendemonstrasikan teknik pengurutan bertingkat (multi-level sorting) menggunakan Java Stream API pada koleksi objek Student. Berikut adalah rincian logikanya:

- Inisialisasi Stream: List students diubah menjadi stream menggunakan metode `.stream()` untuk memulai operasi fungsional.

- Logika Perbandingan (Lambda): Di dalam metode `.sorted()`, digunakan lambda expression yang menampung logika percabangan if-else:
 - Kondisi 1: Program mengecek apakah dua nama siswa sama menggunakan `.equalsIgnoreCase()`. Jika sama, maka urutan ditentukan berdasarkan ID terkecil ke terbesar menggunakan `Integer.compare(s1.getId(), s2.getId())`.
 - Kondisi 2: Jika nama berbeda, program langsung mengurutkan berdasarkan Nama secara alfabetis menggunakan `.compareToIgnoreCase()`.
- Pengumpulan Hasil: Data yang telah terurut dikumpulkan kembali ke dalam sebuah List baru menggunakan `.collect(Collectors.toList())`.

Nama Teman Hal yang Dibantu (Opsional)
-

No. 3 Finding the Nth Largest Element

Soal Praktikum

Skenario

Anda memiliki sebuah array integer primitif. Anda diminta untuk mencari elemen terbesar ke-3 dari array tersebut menggunakan Java 8 Stream. Tahapannya adalah: mengubah array menjadi stream, mengubah tipe data primitif menjadi objek agar bisa diurutkan secara kustom, mengurutkan dari yang terbesar, melompati elemen tertentu, dan mengambil hasilnya.

Instruksi

Isilah bagian rumpang pada kode di bawah ini dengan ketentuan:

1. Bagian [1]: Panggil metode dari kelas Arrays untuk mengubah array menjadi IntStream.
2. Bagian [2]: Gunakan metode untuk mengubah stream primitif (int) menjadi stream objek (Integer) agar bisa menggunakan Comparator.
3. Bagian [3]: Gunakan operasi untuk melompati (skip) sejumlah elemen pertama.
4. Bagian [4]: Gunakan operasi terminal untuk mengambil elemen pertama yang tersedia setelah proses skipping.
5. Bagian [5]: Gunakan metode untuk memberikan nilai default jika stream ternyata kosong.

```
package JavaProblems;
```

```
import java.util.Arrays;
```

```
public class ProblemThree {  
    public static void main(String[] args) {  
        int arr[] = {12, 4, 3, 1, 9, 657};  
        int n = 3; // Mencari elemen terbesar ke-3  
  
        int ans = [1]_____.stream(arr)  
            .[2]_____()           // Konversi ke Stream<Integer>  
            .sorted((a, b) -> Integer.compare(b, a)) // Urutkan descending  
            .[3]_____(n - 1)         // Lewati 2 elemen pertama  
            .[4]_____()           // Ambil elemen yang tersisa  
            .[5]_____(0);            // Nilai default jika tidak ada  
  
        System.out.println("Elemen terbesar ke-3 adalah: " + ans);  
    }  
}
```

Output yang Diharapkan

The 3rd largest element is: 9

Source Code

```

1  package JavaProblems;
2
3  import java.util.Arrays;
4
5  public class ProblemThree {
6      public static void main(String[] args) {
7          int arr[] = {12, 4, 3, 1, 9, 657};
8          int n = 3;
9
10         int ans = Arrays.stream(arr) // [1]
11             .boxed() // [2]
12             .sorted((a, b) -> Integer.compare(b, a))
13             .skip(n - 1) // [3]
14             .findFirst() // [4]
15             .orElse(0); // [5]
16
17         System.out.println("Elemen terbesar ke-3 adalah: " + ans);
18     }
19 }

```

Screenshot Hasil

Elemen terbesar ke-3 adalah: 9

Penjelasan

Program ini bertujuan untuk mencari elemen terbesar ke-3 dari sebuah array integer primitif menggunakan Java 8 Stream API dengan tahapan sebagai berikut:

- Konversi Array ke Stream: Menggunakan `Arrays.stream(arr)` untuk mengubah array `int[]` menjadi `IntStream`.
- Boxing (Primitif ke Objek): Karena `IntStream` (tipe primitif) tidak mendukung pengurutan kustom dengan lambda secara langsung, digunakan metode `.boxed()` untuk mengubahnya menjadi `Stream<Integer>`.
- Pengurutan Menurun (Descending): Menggunakan `.sorted((a, b) -> Integer.compare(b, a))` untuk mengurutkan angka dari yang paling besar ke yang paling kecil.
- Operasi Skipping: Untuk mendapatkan elemen terbesar ke-3 ($n=3$), program menggunakan `.skip(n - 1)` atau `.skip(2)`. Ini berarti dua elemen terbesar pertama (ke-1 dan ke-2) akan dilewati.
- Terminal Operation: Menggunakan `.findFirst()` untuk mengambil elemen pertama yang tersisa setelah proses skipping (yaitu elemen ke-3).
- Handling Nilai Kosong: Digunakan `.orElse(0)` sebagai nilai default seandainya array tersebut kosong, sehingga program tidak mengalami error.

Tracing :

Array semula Adalah {12,4,3,1,9,657}

Sorted (Descending): {657, 12, 9, 4, 3, 1}

Skip(2) : 657 dan 12 dilewati

Maka angka berikutnya Adalah 9, dan `findFirst()` akan menunjuk kesana

Nama Teman Hal yang Dibantu (Opsional)

-